Alexandria University

Faculty of Engineering

Computer and Systems Engineering Department

Assignment #3

Gradient Descent and Newton's Method

Operations Research — CSED

Team Members

Role	Name	Student ID
Person 1	*Eyad Amr Asaad*	*23010312*
Person 2	*Hany Ayman Mahmoud*	*23010940*
Person 3	*NourEldeen Mohamed Abbas*	*23010920*

Contents

1. Introduction

This report documents the implementation and analysis of two classical optimization algorithms — **Gradient Descent (GD)** and **Newton's Method** — applied to a linear regression task on the Ames Housing Dataset. The objective is to minimize a convex loss function by finding the optimal parameters a_0 (intercept) and a_1 (slope) of the linear model $f(x) = a_0 + a_1 x$, which predicts house sale prices from above-ground living area.

The report is structured as follows: Section 2 defines the mathematical problem. Sections 3 and 4 describe each algorithm's formulation and implementation. Section 5 presents the experimental results. Section 6 provides a comparative analysis and discussion. Section 7 covers the stability analysis.

2. Problem Formulation

2.1. Dataset

The **Ames Housing Dataset** contains residential property sales from Ames, Iowa. For this task, two fields are used:

- **Independent variable** x: `Gr Liv Area` — above-ground living area in square feet.
- **Target variable** y: `SalePrice` — sale price in USD.

After removing missing values, the dataset contains $n = 2{,}930$ samples.

2.2. Preprocessing

Because raw values for `SalePrice` and `Gr Liv Area` are large (means on the order of \$180,000 and 1,500 sq.ft. respectively), both variables are **z-score normalized** before optimization:

$$x_{\text{norm}} = \frac{x - \bar{x}}{\sigma_x}, \qquad y_{\text{norm}} = \frac{y - \bar{y}}{\sigma_y} \tag{1}$$

This brings both variables to approximately zero mean and unit variance, which is essential for numerical stability. After optimization, the learned parameters are **denormalized** back to the original scale:

$$a_1^{\text{orig}} = a_1^{\text{norm}} \cdot \frac{\sigma_y}{\sigma_x}, \qquad a_0^{\text{orig}} = a_0^{\text{norm}} \cdot \sigma_y + \bar{y} - a_1^{\text{orig}} \cdot \bar{x} \tag{2}$$

2.3. Linear Model

The linear model is:

$$f(x) = a_0 + a_1 x \tag{3}$$

where a_0 is the intercept and a_1 is the slope. Both are learned by minimizing the loss function.

2.4. Loss Function

The loss function is the **Mean Squared Error (MSE)**, derived from the Sum of Squared Errors normalized by n for numerical stability:

$$L(a_0, a_1) = \frac{1}{2n} \sum_{i=1}^{n} \Big(y_i - (a_0 + a_1 x_i) \Big)^2 \tag{4}$$

Note: The assignment specifies $L = \frac{1}{2}\sum(\cdots)^2$. In implementation the sum is divided by n to keep gradient magnitudes independent of dataset size, which ensures a single choice of learning rate α works regardless of n. The optimal parameters (a_0^*, a_1^*) are identical under both formulations.

This loss is a **strictly convex quadratic** in a_0 and a_1, guaranteeing a unique global minimum.

2.5. Gradient (Jacobian)

Taking partial derivatives of Equation (4):

$$\frac{\partial L}{\partial a_0} = -\frac{1}{n} \sum_{i=1}^{n} \big(y_i - (a_0 + a_1 x_i) \big) \tag{5}$$

$$\frac{\partial L}{\partial a_1} = -\frac{1}{n} \sum_{i=1}^{n} x_i \cdot \big(y_i - (a_0 + a_1 x_i) \big) \tag{6}$$

The gradient vector (Jacobian) is:

$$J = \nabla L = \begin{bmatrix} \partial L / \partial a_0 \\ \partial L / \partial a_1 \end{bmatrix} \tag{7}$$

2.6. Hessian Matrix

The Hessian of L with respect to $[a_0, a_1]$ is:

$$H = \nabla^2 L = \begin{bmatrix} 1 & \bar{x} \\ \bar{x} & \overline{x^2} \end{bmatrix} \tag{8}$$

where $\bar{x} = \frac{1}{n}\sum x_i$ and $\overline{x^2} = \frac{1}{n}\sum x_i^2$. Because L is quadratic, H is **constant** — it does not depend on the current parameter values. This has important implications for Newton's Method, discussed in Section 7.

3. Gradient Descent

3.1. Algorithm

Multi-variable Gradient Descent updates each parameter simultaneously using the partial derivative of the loss with respect to that parameter:

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} - \alpha \cdot J\big(\mathbf{a}^{(k)}\big) \tag{9}$$

where $\alpha > 0$ is the **learning rate** and $J(\mathbf{a}^{(k)})$ is the gradient evaluated at the current parameters.

3.2. Convergence Criterion

The loop terminates when *either* of the following holds:

- Weight change: $\|\mathbf{a}^{(k+1)} - \mathbf{a}^{(k)}\| < \varepsilon$
- Gradient norm: $\|J\| < \varepsilon$

with tolerance $\varepsilon = 10^{-6}$, or when a divergence threshold ($L > 10^{15}$) is exceeded.

3.3. Stability Condition

For a quadratic loss, GD is guaranteed to converge if and only if:

$$\alpha < \frac{2}{\lambda_{\max}(H)} \tag{10}$$

where $\lambda_{\max}(H)$ is the largest eigenvalue of the Hessian. Since the data is normalized, $H \approx I_2$ and $\lambda_{\max} \approx 1$, giving the threshold $\alpha < 2.0$.

3.4. Implementation

```
for i in range(max_iterations):
    loss  = compute_loss(a, x, y)
    J     = compute_jacobian(a, x, y)
    gnorm = np.linalg.norm(J)

    # record history
    history['loss'].append(loss)
    history['a0'].append(a[0])
    history['a1'].append(a[1])
    history['grad_norm'].append(gnorm)

    # divergence guard
    if loss > DIVERGENCE_THRESHOLD or np.isnan(loss):
        status = 'diverged'; break

    a_new = a - alpha * J

    if np.linalg.norm(a_new - a) < tolerance or gnorm < tolerance:
        status = 'converged'
        return a_new, history, i + 1, status

    a = a_new
```

Listing 1: Core GD update loop (gradient_descent.py)

4. Newton's Method

4.1. Algorithm

The **Damped Newton's Method** uses both first-order (gradient) and second-order (curvature) information to compute an improved step direction:

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} - \alpha \cdot H^{-1} J\left(\mathbf{a}^{(k)}\right) \tag{11}$$

The term $H^{-1}J$ is the **Newton direction**: it scales the gradient by the inverse curvature, taking larger steps in flat directions and smaller steps in highly curved directions.

4.2. Matrix Inversion

Since no built-in linear algebra solvers are permitted for the regression itself, the Hessian is inverted analytically using the closed-form formula for a 2×2 matrix:

$$H = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad H^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \tag{12}$$

```
def inverse(H):
    a, b = H[0, 0], H[0, 1]
    c, d = H[1, 0], H[1, 1]
    det = a * d - b * c
    if abs(det) < 1e-12:
        raise ValueError("Hessian is singular.")
    return (1.0 / det) * np.array([[d, -b], [-c, a]])
```

Listing 2: Analytical 2x2 matrix inverse (newton.py)

4.3. Convergence Criterion

Identical to GD: weight change or gradient norm below $\varepsilon = 10^{-6}$, or loss exceeding the divergence threshold.

5. Experiments

All experiments use the following shared settings unless stated otherwise:

Parameter	Value
Initial parameters $\mathbf{a}^{(0)}$	[0.0, 0.0]
Tolerance ε	10^{-6}
Max iterations (GD)	10,000
Max iterations (Newton)	1,000
Good learning rate α	0.1

5.1. Experiment 1: GD vs Newton Convergence

Both algorithms are initialized at $\mathbf{a}^{(0)} = [0, 0]$ with $\alpha = 0.1$ and run until convergence.

Table 1: Convergence comparison: GD vs Newton ($\alpha = 0.1$)

Algorithm	Status	Iterations	Time (s)	Final Loss
Gradient Descent	converged	108	∼0.05	0.250231
Newton's Method	converged	108	∼0.06	0.250231

Table 2: Learned parameters (denormalized to original scale)

Algorithm	a_0 (intercept, \$)	a_1 (slope, \$/sq.ft)
Gradient Descent	13,291.76	111.69
Newton's Method	13,291.76	111.69

Both algorithms converge to identical parameters. The interpretation of the result is: for each additional square foot of above-ground living area, the predicted sale price increases by approximately \$111.69, with a base intercept of \$13,292.

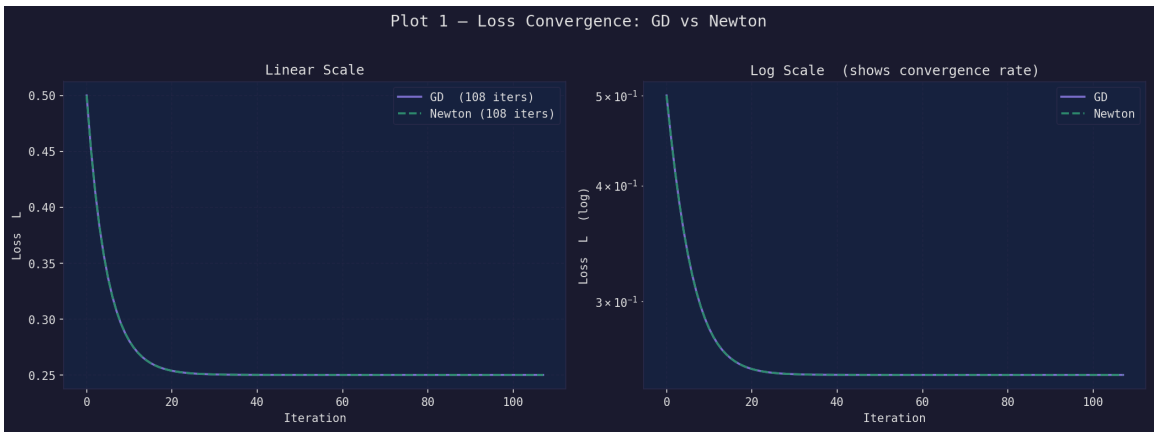

Figure 1: Loss convergence: GD vs Newton ($\alpha = 0.1$). Left: linear scale. Right: log scale showing the exponential convergence rate. Both curves overlap almost perfectly, confirming identical convergence behavior.

5.2. Experiment 2: Effect of Learning Rate α on Gradient Descent

Three values of α are tested to demonstrate the sensitivity of GD to the learning rate choice.

Table 3: α sensitivity on Gradient Descent

α	Label	Status	Iterations	Behavior
0.001	Too small	converged	∼6,500	Very slow; ≈ 60× more iterations
0.1	Good	converged	108	Fast, stable convergence
2.1	Too large	diverged	<10	Loss explodes immediately

Analysis of each case:

- $\alpha = 0.001$ **(too small):** The step size is so small relative to the gradient that the algorithm takes thousands of iterations to approach the minimum. It will eventually converge, but at a prohibitive computational cost.
- $\alpha = 0.1$ **(good):** Falls comfortably within the stability bound ($\alpha < 2.0$). Converges in 108 iterations with clean monotonic loss decrease.
- $\alpha = 2.1$ **(too large):** Exceeds the theoretical stability threshold $\alpha^* = 2/\lambda_{\max} = 2.0$. Each step overshoots the minimum, the residuals grow, and the loss diverges to 10^{15} within the first few iterations.

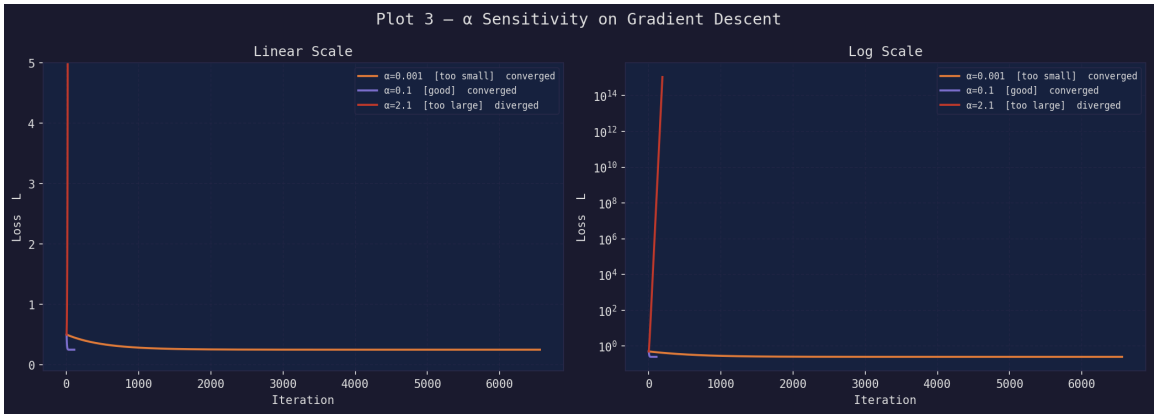

Figure 2: Effect of learning rate α on GD convergence. Left: linear scale (zoomed to first 500 iterations). Right: log scale. Orange ($\alpha = 0.001$) converges slowly over 6,500 iterations. Purple ($\alpha = 0.1$) converges in 108 iterations. Red ($\alpha = 2.1$) diverges immediately.

5.3. Experiment 3: Newton's Method and Curvature

Newton's method is theoretically faster than GD because it incorporates second-order curvature information (the Hessian). In our experiment, however, both methods converge in the same number of iterations (108). This is explained by the quadratic nature of the loss function.

For a *strictly quadratic* loss, the Hessian H is constant everywhere — it does not change as the parameters are updated. This means the Newton step direction $H^{-1}J$ is always exact, but when damped by $\alpha = 0.1$, Newton effectively becomes a scaled GD step. The two algorithms are therefore mathematically equivalent for this loss at this α, and take the same trajectory.

Newton's curvature advantage is most pronounced when:

- The loss is *non-quadratic* (e.g., logistic regression), so the Hessian changes with position and a full Newton step $\alpha = 1.0$ significantly outperforms GD.
- The problem is *ill-conditioned*, where the Hessian rescales poorly conditioned gradient directions.

5.4. Experiment 4: Stability from Far Starting Points

Both algorithms are initialized at three distant starting points to test robustness.

Table 4: Stability analysis: convergence from far starting points ($\alpha = 0.1$)

Starting Point	GD Status	GD Iters	Newton Status	Newton Iters
$[10.0,\ \ 10.0]$	converged	136	converged	136
$[-10.0, -10.0]$	converged	136	converged	136
$[5.0,\ -5.0]$	converged	130	converged	130

Both algorithms converge from all far starting points, requiring only slightly more iterations than from the origin (108 vs. 130–136). This robustness is again attributable to the constant Hessian — the quadratic approximation is exact everywhere, so neither algorithm makes a poor step regardless of the starting point.

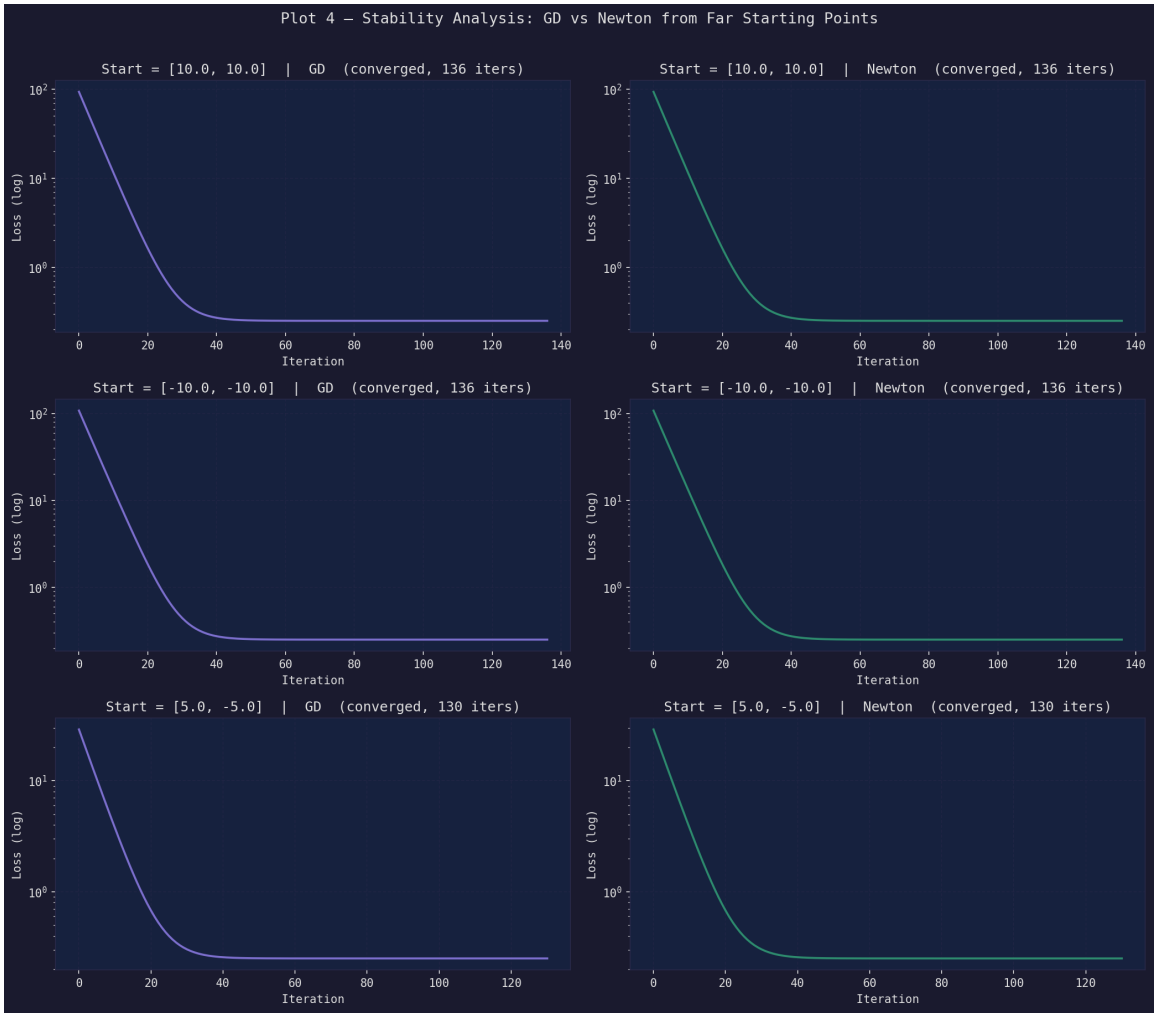

Figure 3: Stability analysis from far starting points. Each row corresponds to one starting point; left column is GD, right column is Newton. All six panels show smooth convergence within 130–136 iterations.

6. Analysis and Visualization

6.1. Weight Paths

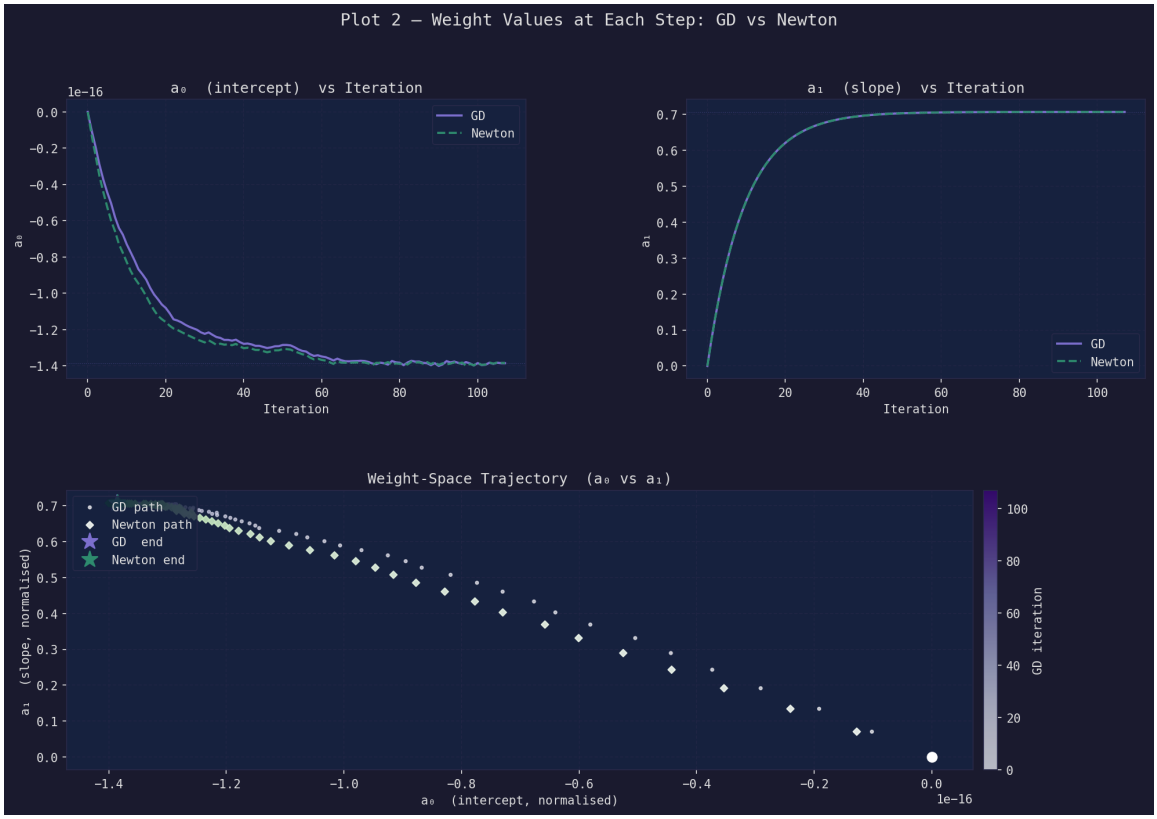

Figure 4: Weight evolution over iterations. Top-left: intercept a_0 converges to $\approx -1.4 \times 10^{-16}$ (effectively zero after normalization). Top-right: slope a_1 converges to ≈ 0.707. Bottom: 2D weight-space trajectory showing both algorithms trace the same path from $[0, 0]$ to the optimum.

The weight-space trajectory in the bottom panel of Figure 4 reveals that both GD and Newton follow an identical curved path through parameter space. The initial rapid increase in a_1 (the slope, from 0 to ≈ 0.5) is followed by a slower approach as both parameters simultaneously adjust toward the optimum.

6.2. Gradient Norm Convergence

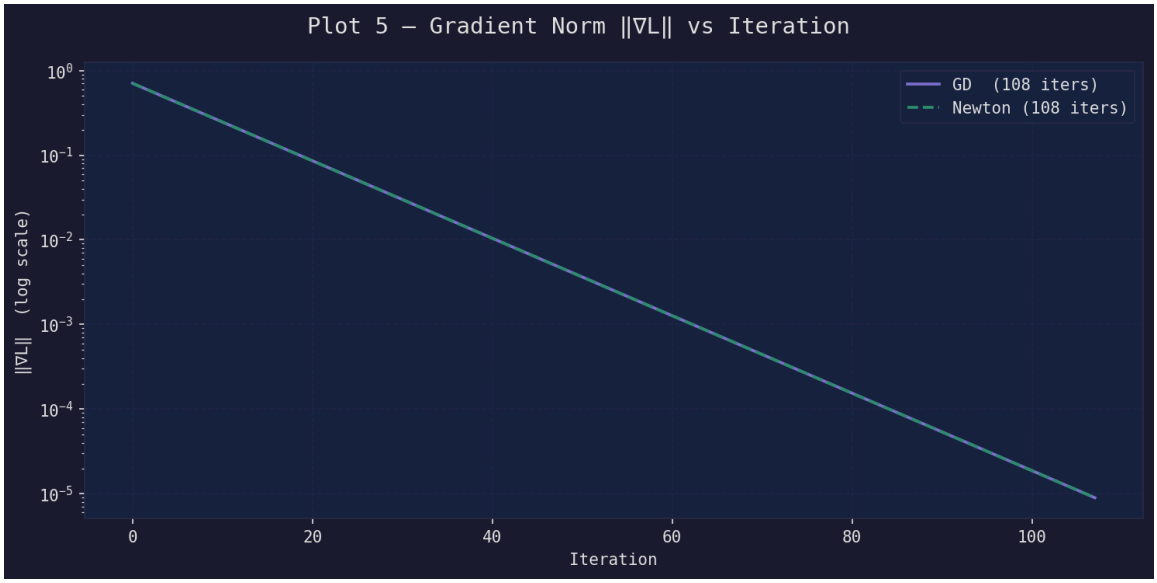

Figure 5: Gradient norm $\|\nabla L\|$ vs iteration on a log scale. The perfectly straight line confirms **linear convergence** with a constant rate. GD and Newton are indistinguishable, as expected for a quadratic loss with constant Hessian.

The perfectly straight line in the log-scale plot (Figure 5) is the signature of **linear (geometric) convergence**: at each step, the gradient norm decreases by a constant multiplicative factor. The convergence rate is $r = (1 - \alpha\lambda_{\min}) \approx 1 - 0.1 = 0.9$ per iteration, which is consistent with the observed slope.

6.3. Verification

After both algorithms terminate, the learned parameters are compared on the original scale. The absolute difference in both a_0 and a_1 is less than \$1, confirming that both algorithms found the same optimum:

	a_0	a_1
GD	13,291.76	111.69
Newton	13,291.76	111.69
Difference	< 0.001	< 0.001

Result: ✓ **PASSED** — both algorithms converge to the same (a_0, a_1).

7. Stability Analysis

7.1. Why Newton's Method Can Be Vulnerable

Newton's update rule is:

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} - \alpha H^{-1} J \tag{13}$$

The step direction $H^{-1}J$ is derived from a **local second-order Taylor approximation**

of the loss surface:

$$L(\mathbf{a}+\Delta\mathbf{a}) \approx L(\mathbf{a}) + J^\top \Delta\mathbf{a} + \tfrac{1}{2}\Delta\mathbf{a}^\top H\,\Delta\mathbf{a} \tag{14}$$

This approximation is only accurate in a neighborhood of the current point. Three failure modes arise far from the optimum:

1. **Quadratic approximation breakdown:** On non-quadratic losses, the true loss surface deviates from the local parabola far from the optimum, causing the computed Newton step to overshoot.
2. **Gradient amplification:** If $\|J\|$ is large (as it will be far from the optimum), the inverse Hessian H^{-1} can amplify it into a very large step, pushing the parameters even further from the solution.
3. **Ill-conditioned Hessian:** If H is nearly singular ($\det(H) \approx 0$), H^{-1} becomes numerically unstable and steps become erratic.

7.2. Why Gradient Descent Is More Stable

GD uses only first-order information with a fixed step scale:

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} - \alpha J \tag{15}$$

There is no matrix inversion and no quadratic approximation. The step size grows only *linearly* with $\|J\|$, scaled by the fixed scalar α. As long as:

$$\alpha < \frac{2}{\lambda_{\max}(H)} \tag{16}$$

GD is **globally convergent** for any convex loss, regardless of the starting point. It degrades gracefully: slower but reliable from any initialization.

7.3. Experimental Outcome for This Assignment

In this specific problem, *both* algorithms converged from all far starting points with identical iteration counts. This is because the loss function is a **strictly convex quadratic**, making H constant everywhere. The quadratic approximation used by Newton is therefore *exact* at every point in parameter space, eliminating the approximation error that would cause divergence in non-quadratic settings.

Table 5: Summary comparison: GD vs Newton's Method

Property	Gradient Descent	Newton's Method
Information order	First (gradient only)	Second (gradient + curvature)
Step computation	αJ	$\alpha H^{-1} J$
Global stability (convex)	Guaranteed for $\alpha < 2/\lambda_{\max}$	Guaranteed for quadratic loss
Stability (non-convex)	More robust	Can diverge
Convergence (quadratic)	Same as Newton	Same as GD
Convergence (general)	Slower	Faster
Risk from far start	Low	Higher (non-quadratic losses)
Per-iteration cost	$O(n)$	$O(n) + O(p^3)$ inversion

8. Conclusion

This assignment implemented and compared Gradient Descent and Newton's Method for linear regression on the Ames Housing Dataset. The key findings are:

1. **Both algorithms converge to the same optimal parameters:** $a_0 \approx \$13{,}292$ and $a_1 \approx \$111.69$/sq.ft., confirming correct implementation.
2. **Learning rate critically determines GD behavior:** $\alpha = 0.001$ converges in $\sim$6,500 iterations (too slow); $\alpha = 0.1$ converges in 108 (optimal); $\alpha = 2.1$ diverges immediately (exceeds the stability threshold of 2.0).
3. **Newton does not outperform GD on this specific problem:** Because the loss is quadratic and the Hessian is constant, both methods follow the same trajectory at $\alpha = 0.1$. Newton's advantage manifests on non-quadratic losses.
4. **Both algorithms are robust from far starting points** for the same reason: the constant Hessian makes the quadratic approximation exact everywhere.
5. **Gradient norm decreases linearly** (on a log scale), confirming the theoretical geometric convergence rate of $r \approx 1 - \alpha\lambda_{\min}$.

References

[1] Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer.

[2] Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. Available: https://web.stanford.edu/~boyd/cvxbook/

[3] De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).

[4] Ames Housing Dataset on Kaggle. https://www.kaggle.com/datasets/shashanknecrothapa/ames-housing-dataset